CSA-1735 Artificial Intelligence

1. Python Program to Solve the 8-Puzzle Problem

Aim: To solve the 8-Puzzle problem using the A* search algorithm.

Algorithm:

1. Define the initial and goal states of the puzzle.
2. Calculate the Manhattan distance heuristic for each state.
3. Select the state with the lowest $f(n) = g(n) + h(n)$.
4. Generate possible states by moving the blank tile up, down, left, or right.
5. Repeat until the goal state is reached.
6. Display the solution path.

Program:

```
import heapq

start = (1, 2, 3,

         4, 0, 6,

         7, 5, 8)

goal = (1, 2, 3,

        4, 5, 6,

        7, 8, 0)

def heuristic(state):

    distance = 0

    for i, tile in enumerate(state):

        if tile != 0:

            j = goal.index(tile)

            distance += abs(i // 3 - j // 3)

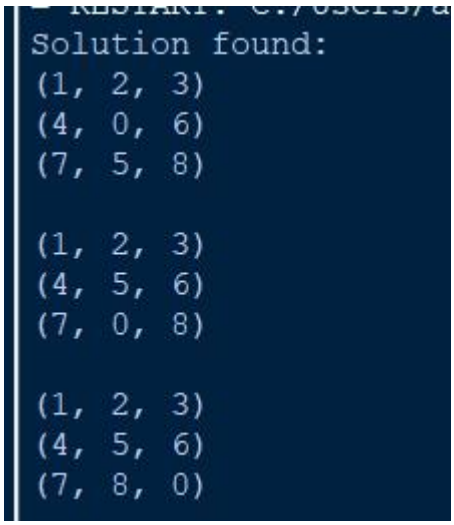
            distance += abs(i % 3 - j % 3)

    return distance
```

```
def solve(start):
    queue = [(heuristic(start), 0, start, [])]
    visited = set()
    while queue:
        _, cost, state, path = heapq.heappop(queue)
        if state in visited:
            continue
        visited.add(state)
        path = path + [state]
        if state == goal:
            return path
        zero = state.index(0)
        row, col = zero // 3, zero % 3
        for dr, dc in [(1, 0), (-1, 0),
                       (0, 1), (0, -1)]:
            nr, nc = row + dr, col + dc
            if 0 <= nr < 3 and 0 <= nc < 3:
                new_pos = nr * 3 + nc
                new_state = list(state)
                new_state[zero], new_state[new_pos] = (
                    new_state[new_pos], new_state[zero]
                )
                new_state = tuple(new_state)
                if new_state not in visited:
                    h = heuristic(new_state)
                    heapq.heappush(
```

```
                    queue,
                    (cost + 1 + h, cost + 1,
                     new_state, path)
                )
    return None
solution = solve(start)
if solution:
    print("Solution found:")
    for state in solution:
        for i in range(0, 9, 3):
            print(state[i:i+3])
        print()
else:
    print("No solution found")
```

Sample Output:



2. Python Program to Solve the 8-Queen Problem

Aim: To place eight queens on an 8 × 8 chessboard so that no two queens attack each other.

Algorithm:

1. Start with an empty chessboard.
2. Place one queen in each row.
3. Check whether the column and diagonals are safe.
4. If safe, place the queen and move to the next row.
5. If no safe position exists, backtrack.
6. Display the solution when all eight queens are placed.

Program:

```
N = 8

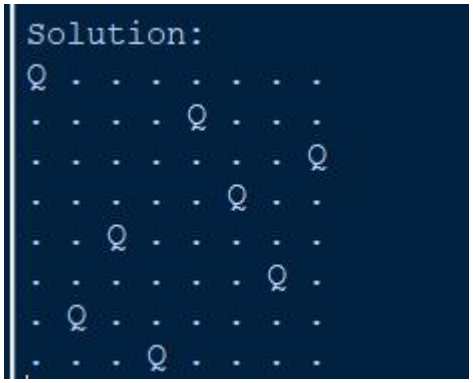
board = [-1] * N

def is_safe(row, col):
    for r in range(row):
        if board[r] == col:
            return False
        if abs(board[r] - col) == abs(r - row):
            return False
    return True
def solve(row):
    if row == N:
        return True
    for col in range(N):
        if is_safe(row, col):
            board[row] = col
            if solve(row + 1):
                return True
            board[row] = -1
    return False
if solve(0):
```

```
    print("Solution:")
    for row in range(N):
        for col in range(N):
            if board[row] == col:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
else:
    print("No solution")
```

Sample Output:

Solution:



3. Python Program for the Water Jug Problem

Aim: To measure exactly 2 litres of water using a 4-litre jug and a 3-litre jug.

Algorithm:

1. Start with both jugs empty.
2. Generate possible actions: fill a jug, empty a jug, or pour water between jugs.
3. Store the visited states to avoid repetition.
4. Use breadth-first search (BFS) to explore the states.
5. Stop when the 4-litre jug contains exactly 2 litres.
6. Display the sequence of states.

Program:

```
from collections import deque

def water_jug():
    start = (0, 0)
    goal = 2

    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        (a, b), path = queue.popleft()

        if a == goal:
            return path

        amount = min(b, 4 - a)
        states = [
            (4, b), (a, 3), (0, b), (a, 0),
            (a + amount, b - amount),
            (a - min(a, 3 - b), b + min(a, 3 - b))
        ]

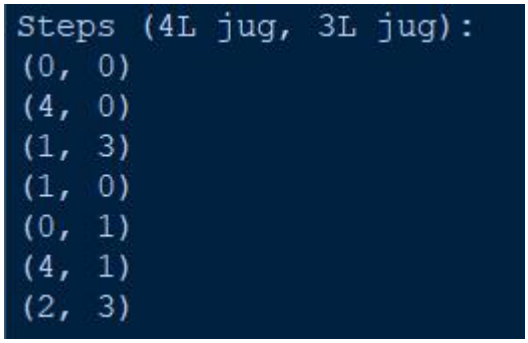
        for state in states:
            if state not in visited:
                visited.add(state)
                queue.append((state, path + [state]))

    return None

result = water_jug()

if result:
    print("Steps (4L jug, 3L jug):")
    for state in result:
        print(state)
else:
    print("Solution not found")
```

Sample Output:



4. Python Program for the Crypt-Arithmetic Problem

Aim: To solve a crypt-arithmetic puzzle in which each letter represents a unique digit. The puzzle used is *SEND + MORE = MONEY*.

Algorithm:

1. Assign a different digit to each letter.
2. Ensure that the first letters of the numbers are not zero.
3. Generate possible digit assignments using backtracking.
4. Replace letters with their assigned digits.
5. Check whether *SEND + MORE = MONEY*.
6. Display the valid digit assignment.

Program:

```
from itertools import permutations


def solve_crypt():
    letters = "SENDMORY"


    for digits in permutations(range(10), len(letters)):
        d = dict(zip(letters, digits))


        if d['S'] == 0 or d['M'] == 0:
            continue
```

```
        send = (d['S'] * 1000 + d['E'] * 100 +
                d['N'] * 10 + d['D'])

        more = (d['M'] * 1000 + d['O'] * 100 +
                d['R'] * 10 + d['E'])

        money = (d['M'] * 10000 + d['O'] * 1000 +
                 d['N'] * 100 + d['E'] * 10 + d['Y'])

        if send + more == money:
            print("SEND =", send)
            print("MORE =", more)
            print("MONEY =", money)
            return

solve_crypt()
```

Sample Output:

```
SEND = 9567
MORE = 1085
MONEY = 10652
```

Result: The puzzle is solved because *9567 + 1085 = 10652*.

5. Python Program for the Missionaries and Cannibals Problem

Aim: To transport three missionaries and three cannibals across a river using a boat that can carry at most two people, without allowing cannibals to outnumber missionaries on either bank when missionaries are present.

Algorithm:

1. Represent the state as (M, C, B), where M and C are the missionaries and cannibals on the left bank, and B indicates the boat position.
2. Start with $(3, 3, 1)$.
3. Generate boat movements carrying one or two people.
4. Check that the number of people remains valid on both banks.
5. Use BFS to find a sequence of valid states.
6. Display the solution path until everyone reaches the right bank.

Program:

```
from collections import deque

def valid(m, c):
    mr, cr = 3 - m, 3 - c

    return (0 <= m <= 3 and 0 <= c <= 3 and
            (m == 0 or m >= c) and
            (mr == 0 or mr >= cr))

def solve():
    start = (3, 3, 1)
    goal = (0, 0, 0)

    moves = [(1, 0), (2, 0), (0, 1),
             (0, 2), (1, 1)]

    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        state, path = queue.popleft()
        m, c, boat = state

        if state == goal:
            return path
```

```
        for dm, dc in moves:
            if boat == 1:
                new_state = (m - dm, c - dc, 0)
            else:
                new_state = (m + dm, c + dc, 1)

            nm, nc, nb = new_state

            if valid(nm, nc) and new_state not in visited:
                visited.add(new_state)
                queue.append((new_state, path + [new_state]))

    return None

path = solve()

if path:
    print("Solution path (M, C, Boat):")
    for state in path:
        print(state)
else:
    print("No solution found")
```

Sample Output:

```
Solution path (M, C, Boat):
(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)
```

Result: All three missionaries and three cannibals are transported safely to the opposite bank.